

Un test en programación es el proceso de verificar y validar que una aplicación funcione según lo esperado (según sus requerimientos). Este proceso se realiza ejecutando código para probar el sistema, encontrar errores (bugs) y asegurar la calidad. Su objetivo es confirmar que el software cumple con los requerimientos funcionales y técnicos, evitando fallos antes del lanzamiento.

Esta tutoría está diseñada para que comprendas cómo funciona el "laboratorio" de pruebas que hemos construido. No se trata solo de escribir código, sino de entender cómo simulamos el comportamiento de un usuario para verificar que el servidor responda correctamente.

1. El motor de pruebas: `testUtils.js`

<https://github.com/gabrielinuz/sample-vault-tests/blob/main/frontend/js/tests/testUtils.js>

Este objeto es nuestro "kit de herramientas" para interactuar con el DOM (la interfaz de la página). Su objetivo es que no tengas que preocuparte por cómo crear botones o cómo escribir en pantalla, sino solo por la lógica del test.

El ciclo de vida de un botón de prueba

Cuando usas `testUtils.createTestButton(label, testFn)`, ocurren tres estados visuales que debes conocer:

- **Gris (Reposo)**: El botón está listo para ser presionado.

- **Amarillo (Cargando):** Al hacer clic, el botón cambia a amarillo. Esto sucede porque usamos `await testFn(btn)`. El sistema espera a que la petición `fetch` regrese del servidor antes de seguir.
- **Verde o Rojo (Resultado):**
 - Si la lógica dentro de tu test llama a `testUtils.setSuccess(btn)`, se pone **verde**.
 - Si ocurre un error inesperado (el servidor está caído, error de sintaxis), el `catch` lo captura, muestra el error en la consola y pone el botón en **rojo**.

La Consola Simulada: `log(data)`

La función `log` toma cualquier objeto (normalmente la respuesta `data` del servidor), lo convierte en un texto legible usando `JSON.stringify(data, null, 2)` y lo inyecta en el elemento `api-console`. Esto es vital para "ver" qué está pensando el servidor.

2. Independencia y el rol de `okLogin()`

Uno de los principios del testing es la **independencia**. Un test no debería fallar solo porque olvidaste ejecutar otro test antes. Aquí es donde entra la función asincrónica `okLogin()`.

¿Por qué usar `async/await` aquí?

El servidor tarda tiempo en validar las credenciales. Si no usáramos `await okLogin()`, el código intentaría pedir la lista de samples antes de tener el token, lo que resultaría en un error `401 Unauthorized`.

```

async function okLogin()
{
  // 1. Login como productor (pepe) para obtener un token válido
  const response = await fetch('/api/auth/login', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ username: 'pepe', password: '12345'
  }) // Usamos pepe hardcodeado
  });
  const data = await response.json();
  // Guardamos el token para tests de samples
  localStorage.setItem('test_token', data.token);
}

```

Al guardar el token en el `localStorage`, permitimos que cualquier test posterior lo recupere para identificarse ante la API.

3. Anatomía de un Test: Petición y Validación

Todos los tests que verás en `authTests.js` o `sampleTests.js` siguen esta estructura de tres pasos:

1. **Preparación:** Obtener el token (si la ruta es protegida) y preparar los datos a enviar (`JSON` o `FormData`).
2. **Ejecución:** Llamar a `fetch` con el método correcto (`GET`, `POST`, `DELETE`).
3. **Verificación:**
 - Transformar la respuesta a JSON: `await response.json()`.
 - Mostrar el resultado: `testUtils.log(data)`.
 - Validar el estado: Si `response.ok` es verdadero (código 200-299), marcamos éxito.

4. Casos Especiales: `FormData` y `Blob`

En `sampleTests.js`, verás algo diferente: el uso de `FormData`. Esto es necesario porque para subir archivos no podemos enviar un simple JSON.

- **FormData:** Es un objeto que simula un formulario de HTML. Nos permite adjuntar archivos y campos de texto mezclados.
- **Blob:** Como no queremos seleccionar un archivo real de tu disco duro cada vez que corremos el test, creamos un "archivo falso" en memoria:

```
const blob = new Blob(["Contenido de audio"], { type: 'audio/wav' });
formData.append('audioFile', blob, 'test.wav'); // Simulamos la subida
```

Consejos para tus Ejercicios

- **Para el Ej01 (Registro):** No necesitas token. Es un test "público". Solo asegúrate de que el `username` sea diferente cada vez que presiones el botón (por eso usamos `Date.now()`).
- **Para el Ej02 (Seguridad):** Aquí buscas que falle. Si `response.status === 403`, entonces tu código de seguridad en el backend está funcionando bien.
- **Para el Ej03 (Borrado Dinámico):** Primero haz un `GET` a `/api/samples/my-samples`. Mira la respuesta en la consola, identifica cómo se llama la propiedad del ID (normalmente `samples[0].id`) y úsala para armar la URL del `DELETE`.
- **Para el Ej04 (Validación):** Intenta enviar el `FormData` vacío. El objetivo es ver cómo reacciona tu servidor cuando un usuario intenta "romper" las reglas